

Detail mathematique du pipeline d'analyse DXD

1 Objet du document

Ce document decrit les calculs effectues par les scripts du depot, de la lecture des fichiers DXD jusqu'aux index exploites par l'application. Il suit principalement le flux moderne :

1. `00_scan_dxd_channel_presence.py` : lecture DXD, resampling, export JSON canonique, features globales et glissantes.
2. `build_window_quality_index.py` : calcul par fenetre des residus physiques, scores qualite et labels de contexte.
3. `build_window_cluster_index.py` : regroupement DBSCAN de fenetres dynamiquement comparables.
4. `build_speed_day_summary.py`, `build_drift_index.py`, `build_correlation_anomaly_index.py` : agregations et index secondaires.

Les scripts `01_export_dxd_resampled_json.py` et `02_extract_features.py` constituent un flux historique ou auxiliaire. Leurs calculs sont resumes en fin de document.

2 Notations generales

Pour une acquisition, les signaux sont resamples sur une grille temporelle commune :

$$t_i = t_{\min} + i\Delta t, \quad i = 0, \dots, N - 1.$$

Dans la configuration par default de l'export DXD :

$$f_s = 100 \text{ Hz}, \quad \Delta t = \frac{1}{f_s} = 0.01 \text{ s}.$$

Les grandeurs principales sont :

$v_x(t)$: vitesse longitudinale vehicule en m/s,

$v_y(t)$: vitesse laterale vehicule en m/s,

$a_x(t)$: acceleration longitudinale en m/s²,

$a_y(t)$: acceleration laterale en m/s²,

$r(t)$: vitesse de lacet, ou yaw rate, en rad/s,

$\omega_{fl}, \omega_{fr}, \omega_{rl}, \omega_{rr}$: vitesses angulaires des roues.

Les valeurs non finies (NaN, infinis) sont ignorees dans les moyennes, RMS et quantiles lorsque le code utilise les fonctions `safe_*`. On note :

$$\mathcal{F}(x) = \{i \mid x_i \text{ est fini}\}.$$

Pour un vecteur x , les statistiques de base utilisees sont :

$$\bar{x} = \frac{1}{|\mathcal{F}(x)|} \sum_{i \in \mathcal{F}(x)} x_i, \quad \text{RMS}(x) = \sqrt{\frac{1}{|\mathcal{F}(x)|} \sum_{i \in \mathcal{F}(x)} x_i^2},$$
$$\max |x| = \max_{i \in \mathcal{F}(x)} |x_i|, \quad q_{95}(|x|) = \text{quantile a 95\% de } |x_i|.$$

La derivee numerique est calculee par gradient sur les points finis :

$$\dot{x}_i \simeq \nabla x_i / \Delta t.$$

3 Normalisation des canaux

Le module `dxd_schema.py` transforme les noms sources Dewesoft en noms canoniques. Exemples :

Nom source	Nom canonique	Conversion
VelX (km/h)	<code>vehicle.vx</code>	$v_x = \text{VelX}/3.6$
VelY (km/h)	<code>vehicle.vy</code>	$v_y = \text{VelY}/3.6$
Vel (km/h)	<code>vehicle.speed</code>	$v = \text{Vel}/3.6$
VehYaw_W_Act1 (rad/s)	<code>vehicle.yaw_rate</code>	identite
VehLong_A_Act1 (m/s ²)	<code>vehicle.ax</code>	identite
VehLat_A_Act1 (m/s ²)	<code>vehicle.ay</code>	identite
Latitude (_)	<code>gps.latitude</code>	identite
Longitude (_)	<code>gps.longitude</code>	identite

En general, si un canal source y^{raw} a une echelle s et un offset b , le canal canonique vaut :

$$y = s y^{raw} + b.$$

4 Lecture DXD, nettoyage et resampling

Pour chaque canal lu dans un fichier DXD, le code :

1. extrait les couples (t_i, y_i) ;
2. supprime les couples non finis ;
3. trie par temps croissant ;
4. supprime les doublons temporels en gardant la premiere occurrence ;
5. calcule l'intersection temporelle commune a tous les canaux selectionnes :

$$t_{\min} = \max_c t_0^{(c)}, \quad t_{\max} = \min_c t_{\text{fin}}^{(c)}.$$

La grille commune est :

$$t_k = t_{\min} + k\Delta t, \quad t_k \leq t_{\max} + 0.5\Delta t.$$

Chaque canal est interpole lineairement :

$$y_c(t_k) = \text{interp}(t_k; t^{(c)}, y^{(c)}),$$

avec NaN a gauche et a droite hors du support du canal.

5 Canaux derives dans l'export JSON

Le script `00_scan_dxd_channel_presence.py` ajoute certains residus derives au JSON. Ces residus sont calcules sur les noms sources avant normalisation complete.

5.1 Coherence laterale simple

Si a_y , v_x et r sont disponibles :

$$e_{lat}(t) = a_y(t) - v_x(t)r(t).$$

Le JSON contient aussi un RMS glissant sur une fenetre de duree $T_r = 2$ s par default :

$$S_{lat}(i) = \sqrt{\frac{1}{|\mathcal{W}_i|} \sum_{j \in \mathcal{W}_i} e_{lat}(j)^2},$$

ou $\mathcal{W}_i$ est la fenetre retrospective de longueur T_r finissant en i .

5.2 Differences de vitesses de roues

Les residus de roues exportes dans le JSON sont :

$$\begin{aligned} e_{front}(t) &= \omega_{fl}(t) - \omega_{fr}(t), \\ e_{rear}(t) &= \omega_{rl}(t) - \omega_{rr}(t), \\ e_{left}(t) &= \omega_{fl}(t) - \omega_{rl}(t), \\ e_{right}(t) &= \omega_{fr}(t) - \omega_{rr}(t). \end{aligned}$$

Chaque residu possede egalement un RMS glissant sur T_r .

5.3 Steer, camber et acceleration longitudinale

Pour les deux voies de direction et de carrossage :

$$e_\delta(t) = \delta_{S1}(t) + \delta_{S2}(t), \quad e_\gamma(t) = \gamma_{S1}(t) - \gamma_{S2}(t).$$

Pour l'acceleration longitudinale :

$$e_{ax}(t) = a_x(t) - \frac{dv_x}{dt}(t).$$

6 Features globales et glissantes dans le JSON

Pour chaque canal brut ou derive $y(t)$, le JSON contient des features globales :

$$n_{valid}, \quad \bar{y}, \quad \sigma_y, \quad \min y, \quad \max y, \quad \text{RMS}(y), \quad E_y.$$

L'energie est :

$$E_y = \sum_{i \in \mathcal{F}(y)} y_i^2 \Delta t.$$

Les features glissantes sont calculees par fenetres de duree $T_w = 4$ s et pas $T_s = 1$ s par default :

$$\mathcal{W}_m = \{i_m, \dots, i_m + n_w - 1\}, \quad n_w = \text{round}(T_w/\Delta t).$$

Pour chaque fenetre, le code stocke :

$$\bar{y}_{\mathcal{W}}, \quad \sigma_{y,\mathcal{W}}, \quad \text{RMS}(y_{\mathcal{W}}), \quad \min_{\mathcal{W}} y, \quad \max_{\mathcal{W}} y.$$

Quelques correlations globales sont aussi calculees sur des paires fixes :

$$\rho(x, y) = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}.$$

7 Fenetrage pour l'index qualite

Le script `build_window_quality_index.py` lit les JSON resamples et reconstruit des fenetres de duree $T_w = 4$ s avec un pas $T_s = 1$ s :

$$n_w = \max(2, \text{round}(T_w/\Delta t)), \quad n_s = \max(1, \text{round}(T_s/\Delta t)).$$

Les fenetres sont :

$$\mathcal{W}_m = \{mn_s, \dots, mn_s + n_w - 1\}.$$

Pour chaque fenetre, les statistiques dynamiques principales sont :

$$\bar{v}_x, \quad \bar{v}_x^{km/h} = 3.6 \bar{v}_x, \quad \text{RMS}(a_x), \quad \text{RMS}(a_y), \quad \text{RMS}(r), \quad \text{RMS}(\delta).$$

Deux ratios de vitesse sont calcules :

$$p_{zero} = \frac{1}{|\mathcal{F}(v_x)|} \sum_{i \in \mathcal{F}(v_x)} \mathbf{1}(|v_x(i)| < 1/3.6),$$

$$p_{low} = \frac{1}{|\mathcal{F}(v_x)|} \sum_{i \in \mathcal{F}(v_x)} \mathbf{1}(|v_x(i)| < 5/3.6).$$

8 Qualite de signal par canal

Pour chaque canal de qualite, dans une fenetre $\mathcal{W}$:

$$r_{finite} = \frac{\#\{i \in \mathcal{W} \mid y_i \text{ fini}\}}{|\mathcal{W}|}.$$

Le score de presence vaut :

$$s_{finite} = \begin{cases} 1.0 & \text{si } r_{finite} \geq 0.95, \\ 0.5 & \text{si } 0.80 \leq r_{finite} < 0.95, \\ 0.0 & \text{sinon.} \end{cases}$$

Les spikes sont detectes sur les differences successives des valeurs finies :

$$d_j = y_{j+1} - y_j.$$

Si $\sigma_d > 10^{-12}$, le nombre de spikes est :

$$n_{spike} = \#\{j \mid |d_j| > 6\sigma_d\}.$$

La penalite est :

$$p_{spike} = \min\left(0.5, \frac{n_{spike}}{|\mathcal{W}|}\right).$$

Le score canal est :

$$s_{canal} = \max(0, s_{finite} - p_{spike}).$$

Le score global de qualite signal est la moyenne des scores canaux disponibles :

$$S_{signal} = \text{mean}\{s_{canal,c}\}.$$

9 Residus physiques de l'index qualite

Les residus suivants sont calcules sur toute l'acquisition, puis resumes par fenetre via moyenne, RMS, maximum absolu et quantile absolu a 95%.

9.1 Cinematique longitudinale

$$r_{ax}(t) = a_x(t) - \frac{dv_x}{dt}(t).$$

9.2 Cinematique laterale

Mode simple :

$$r_{ay,simple}(t) = a_y(t) - v_x(t)r(t).$$

Mode avec vitesse laterale :

$$r_{ay}(t) = a_y(t) - \left(\frac{dv_y}{dt}(t) + v_x(t)r(t)\right).$$

9.3 Distance GPS

Si une distance GPS curviligne $d_{gps}(t)$ est disponible, avec $v_{ref} = v$ si la vitesse scalaire est disponible, sinon v_x :

$$r_{dist}(t) = \frac{dd_{gps}}{dt}(t) - v_{ref}(t).$$

9.4 Courbure

Si le rayon GPS R_{gps} est disponible :

$$\kappa_{gps}(t) = \frac{1}{R_{gps}(t)}.$$

La courbure vehicule est :

$$\kappa_{veh}(t) = \begin{cases} \frac{r(t)}{\max(|v_x(t)|, 5/3.6)} & \text{si } |v_x(t)| \geq 5/3.6, \\ \text{NaN} & \text{sinon.} \end{cases}$$

Le residu est :

$$r_{\kappa}(t) = \kappa_{gps}(t) - \kappa_{veh}(t).$$

9.5 Roues

Si les quatre vitesses de roues sont disponibles :

$$r_{wheel}(t) = \text{std}(\omega_{fl}(t), \omega_{fr}(t), \omega_{rl}(t), \omega_{rr}(t)).$$

9.6 Calibration IMU

$$r_{imu,x}(t) = a_{imu,x}(t) - a_x(t), \quad r_{imu,y}(t) = a_{imu,y}(t) - a_y(t), \quad r_{imu,z}(t) = a_{imu,z}(t) - a_z(t).$$

10 Residus GPS

10.1 Projection latitude/longitude en repere local

Les latitudes et longitudes valides sont filtrees. Avec le premier point valide comme origine (φ_0, λ_0) , et $R_T = 6\,371\,000$ m :

$$x_{gps} = R_T \cos(\varphi_0)(\lambda - \lambda_0), \quad y_{gps} = R_T(\varphi - \varphi_0),$$

ou les angles sont en radians.

10.2 Vitesse GPS

Les positions GPS sont lissees par moyenne mobile d'environ 0.5 s, puis derivees :

$$v_{gps,x} = \frac{dx_{gps}}{dt}, \quad v_{gps,y} = \frac{dy_{gps}}{dt},$$

$$v_{gps} = \sqrt{v_{gps,x}^2 + v_{gps,y}^2}.$$

Les residus de vitesse GPS sont :

$$r_{gps,speed} = v_{gps} - v_{ref},$$

$$r_{gps,body} = v_{gps} - \sqrt{v_x^2 + v_y^2},$$

ou, si v_y est absent, $\sqrt{v_x^2 + v_y^2}$ est remplace par $|v_x|$.

10.3 Integration trajectoire vehicule

L'angle de cap initial ψ_0 vient d'abord de la trajectoire GPS dans la fenetre si le deplacement total depasse 5 m, sinon du canal `gps.track`, sinon d'une derivee de trajectoire, sinon 0.

Le yaw rate est integre :

$$\psi_i = \psi_0 + \sum_{k=0}^i r_k \Delta t.$$

La vitesse corps est projetee dans le repere monde :

$$\dot{x}_{veh} = v_x \cos \psi - v_y \sin \psi, \quad \dot{y}_{veh} = v_x \sin \psi + v_y \cos \psi.$$

Puis :

$$x_{veh,i} = x_{veh,0} + \sum_{k=1}^i \dot{x}_{veh,k} \Delta t, \quad y_{veh,i} = y_{veh,0} + \sum_{k=1}^i \dot{y}_{veh,k} \Delta t.$$

L'erreur de position brute est :

$$r_{gps,pos}(i) = \sqrt{(x_{gps,i} - x_{veh,i})^2 + (y_{gps,i} - y_{veh,i})^2}.$$

10.4 Erreur GPS alignee

Pour une fenetre locale, le code centre la trajectoire GPS et la trajectoire integree, puis cherche une rotation 2D de type Procrustes. Si p_i est la position GPS centree et q_i la position integree centree :

$$d = \sum_i q_{x,i} p_{x,i} + q_{y,i} p_{y,i}, \quad c = \sum_i q_{x,i} p_{y,i} - q_{y,i} p_{x,i},$$

$$\theta = \text{atan2}(c, d).$$

La trajectoire integree est tournee de θ , recentree sur le GPS, puis l'erreur alignee vaut :

$$r_{gps,pos,aligned}(i) = \|p_i - \tilde{q}_i\|_2.$$

10.5 Yaw rate GPS

Si `gps.track` existe, le cap GPS est unwrappe et derive :

$$r_{gps,yaw}(t) = \frac{d\psi_{gps}}{dt}(t) - r(t),$$

uniquement lorsque la vitesse corps est superieure au seuil basse vitesse 5/3.6 m/s.

11 Transformation residu vers score

Chaque residu RMS peut etre transforme en score par :

$$s(e; \tau) = \frac{1}{1 + \max(e, 0)/\tau}.$$

Un residu nul donne 1. Un residu egal a τ donne 0.5.

Les seuils τ utilises sont :

Score	Residus et seuils
<code>kinematic_score</code>	r_{dist} avec $\tau = 1.5$, r_{κ} avec $\tau = 0.05$
<code>acceleration_score</code>	r_{ax} avec $\tau = 1.0$, r_{ay} et $r_{ay,simple}$ avec $\tau = 1.5$

gps_score	$r_{gps,pos}$ avec $\tau = 10$, $r_{gps,pos,aligned}$ avec $\tau = 5$, $r_{gps,speed}$ et $r_{gps,body}$ avec $\tau = 2$, $r_{gps,yaw}$
wheel_score	r_{wheel} avec $\tau = 1.0$
calibration_score	$r_{imu,x}$, $r_{imu,y}$, $r_{imu,z}$ avec $\tau = 1.0$

Les moyennes ignorent les scores absents :

$$S_{phys} = \text{mean}(S_{kin}, S_{acc}, S_{gps}, S_{wheel}, S_{calib}),$$

$$S_{global} = \text{mean}(S_{signal}, S_{phys}).$$

Important : dans le code actuel, S_{global} est un score absolu par fenetre. Il n'est pas normalise par cluster. Le clustering ajoute un contexte de comparaison mais ne modifie pas ce score.

12 Scores d'excitation et flags d'usage

Les scores d'excitation sont :

$$S_{lat_exc} = \min\left(1, \frac{\text{RMS}(a_y)}{3}\right), \quad S_{long_exc} = \min\left(1, \frac{\text{RMS}(a_x)}{2}\right).$$

Les flags d'usage sont :

$$\text{usable_for_regression} = (S_{global} \geq 0.65) \wedge (p_{low} < 0.5),$$

$$\text{usable_for_lateral_identification} = \text{usable_for_regression} \wedge (S_{lat_exc} \geq 0.25),$$

$$\text{usable_for_longitudinal_identification} = \text{usable_for_regression} \wedge (S_{long_exc} \geq 0.25),$$

$$\text{usable_for_tire_parameter_estimation} = \text{usable_for_lateral_identification} \wedge (r_{wheel} \text{ a le statut compu}$$

$$\text{usable_for_indicator_computation} = (S_{signal} \geq 0.8).$$

13 Bins et familles de manoeuvre

Les bins vitesse sont construits sur $\overline{v_x}^{km/h}$:

$$[0, 20), [20, 40), [40, 60), [60, 80), [80, 100), [100, 120),$$

sinon **out_of_range** ou **unknown**.

Les bins d'acceleration laterale :

$$\begin{cases} \text{faible} & \text{RMS}(a_y) < 1, \\ \text{moyen} & 1 \leq \text{RMS}(a_y) < 3, \\ \text{fort} & 3 \leq \text{RMS}(a_y) < 6, \\ \text{tres_fort} & \text{RMS}(a_y) \geq 6. \end{cases}$$

Les bins d'acceleration longitudinale :

$$\begin{cases} \text{faible} & \text{RMS}(a_x) < 0.5, \\ \text{moyen} & 0.5 \leq \text{RMS}(a_x) < 2, \\ \text{fort} & \text{RMS}(a_x) \geq 2. \end{cases}$$

La direction de manoeuvre est determinee par le yaw rate :

$$f_+ = \text{Pr}(r > 0.03), \quad f_- = \text{Pr}(r < -0.03).$$

Si $f_+ > 0.2$ et $f_- > 0.2$, la direction est **mixed**. Sinon le signe de la moyenne de r donne **left**, **right** ou **straight**.

La famille de manoeuvre suit les regles :

stopped	$\overline{v_x}^{km/h} < 1$,
straight	$\overline{v_x}^{km/h} \geq 5$, $\text{RMS}(a_y) < 0.5$, $\text{RMS}(r) < 0.03$, $\text{RMS}(\delta) < 0.02$,
lateral	$\text{RMS}(a_y) \geq 1$, $\text{RMS}(r) \geq 0.05$, $\text{RMS}(a_x) < 1$,
longitudinal	$\text{RMS}(a_x) \geq 1$, $\text{RMS}(a_y) < 1$,
combined	$\text{RMS}(a_x) \geq 1$, $\text{RMS}(a_y) \geq 1$,
unknown	sinon.

L'identifiant de groupe de similarite est une concatenation :

`family__v_speedBin__ay_latBin__steer_steerBin__dir_direction.`

14 Clustering de fenetres comparables

Le script `build_window_cluster_index.py` lit `window_quality_index.csv`. Les features utilisees pour clusteriser sont :

$$z = (\overline{v_x}^{km/h}, \text{RMS}(a_x), \text{RMS}(a_y), \text{RMS}(r), \text{RMS}(\delta), \\ p_{zero}, p_{low}, S_{lat_exc}, S_{long_exc}).$$

Seules les colonnes disponibles sont gardees. Il faut au moins trois features.

Le clustering est fait separement par valeur de `maneuver_family`, si le groupe contient au moins `min_group_size=30` fenetres.

14.1 Standardisation robuste

Pour chaque feature x dans un groupe :

$$m = \text{median}(x), \quad q_{25} = Q_{0.25}(x), \quad q_{75} = Q_{0.75}(x), \\ s = q_{75} - q_{25}.$$

Si s est nul ou non fini, le code utilise l'ecart-type ; si celui-ci est aussi nul, $s = 1$. Les valeurs manquantes sont remplacees par la mediane, puis :

$$\tilde{x}_i = \frac{x_i - m}{s}.$$

14.2 DBSCAN

Dans l'espace standardise $\tilde{z}$, le voisinage de rayon $\varepsilon = 1.35$ est :

$$\mathcal{N}_\varepsilon(i) = \{j \mid \|\tilde{z}_j - \tilde{z}_i\|_2 \leq \varepsilon\}.$$

Un point est coeur si :

$$|\mathcal{N}_\varepsilon(i)| \geq \text{min_samples} = 18.$$

Les clusters sont les composantes connectees atteignables depuis les points coeurs. Les points non atteints restent **noise**.

14.3 Distance au centre de cluster

Pour chaque cluster C , le centre robuste est :

$$c_C = \text{median}\{\tilde{z}_i \mid i \in C\}$$

calcule composante par composante. La distance stockee est :

$$d_i = \|\tilde{z}_i - c_C\|_2.$$

Le fichier `window_cluster_index.csv` reprend toutes les colonnes de `window_quality_index.csv` et ajoute notamment :

`cluster_id`, `cluster_label`, `cluster_is_noise`, `cluster_size`, `cluster_distance_to_center`.

15 Agregation par date et vitesse

Le script `build_speed_day_summary.py` groupe les fenetres par :

$$(\text{date}, \text{speed_bin}).$$

Pour chaque colonne numerique x , il calcule :

$$\text{mean}(x), \quad \text{median}(x), \quad Q_{0.95}(x).$$

Pour chaque flag boolean b , il calcule le ratio :

$$\text{ratio}(b) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(b_i = \text{True}).$$

Il stocke aussi le nombre de fenetres, le nombre de fichiers et la famille de manoeuvre dominante.

16 Index de drift

`build_drift_index.py` ne recalcule pas de nouvelle metrique physique. Il lit les JSON exportes, extrait les features globales par canal et les regroupe dans `drift_index.json`. La logique mathematique est donc celle des features globales de la section precedente.

17 Index d'anomalies de correlation

Le script `build_correlation_anomaly_index.py` exploite les moyennes de fenetres glissantes des JSON :

$$x_w = \text{mean}_{\mathcal{W}}(x), \quad y_w = \text{mean}_{\mathcal{W}}(y).$$

17.1 Reference robuste par piste

Les fichiers sont regroupes par `track_id`, derive du nom de fichier. Pour chaque paire de capteurs (x, y) et chaque piste, le code collecte les couples (x_w, y_w) .

Les echelles robustes sont :

$$\begin{aligned} m_x &= \text{median}(x_w), & s_x &= 1.4826 \text{ median}(|x_w - m_x|), \\ m_y &= \text{median}(y_w), & s_y &= 1.4826 \text{ median}(|y_w - m_y|). \end{aligned}$$

Les valeurs standardisees sont :

$$z_x = \frac{x_w - m_x}{s_x}, \quad z_y = \frac{y_w - m_y}{s_y}.$$

La correlation de reference est :

$$\rho = \text{corr}(z_x, z_y).$$

Le residu relationnel est :

$$r_{rel} = \frac{z_y - \rho z_x}{\sqrt{\max(1 - \rho^2, 0.10)}}.$$

Sa reference robuste est :

$$m_r = \text{median}(r_{rel}), \quad s_r = \max(1.4826 \text{ median}(|r_{rel} - m_r|), 0.50).$$

17.2 Score d'anomalie relationnelle

Pour une fenetre donnee :

$$A = \frac{|r_{rel} - m_r|}{s_r}.$$

Une anomalie candidate est gardee si :

$$A \geq 2.5.$$

La severite est :

$$\begin{cases} \text{high} & A \geq 5.0, \\ \text{medium} & 3.5 \leq A < 5.0, \\ \text{low} & A < 3.5. \end{cases}$$

18 Flux historique : export 01 et features 02

18.1 Export historique 01

`01_export_dxd_resampled_json.py` selectionne seulement les fichiers qui possedent tous les canaux cibles, construit une grille a 100 Hz, interpole les canaux et exporte un JSON canonique. Il ne calcule pas les residus physiques avances de l'index qualite.

18.2 Features univariees 02

`02_extract_features.py` construit des fenetres de 1 s avec un pas de 1 s par default. Les canaux sont gardes si leur ratio global de valeurs finies est au moins 0.90 et s'il reste au moins trois canaux.

Dans chaque fenetre, les valeurs manquantes de chaque canal sont interpolées. Si un canal est entierement manquant dans une fenetre, la fenetre est rejetee.

Pour chaque canal y , les features sont :

$$\bar{y}, \sigma_y, \min y, \max y, \max y - \min y, \text{median}(y), \\ Q_{0.25}(y), Q_{0.75}(y), \text{IQR}(y), \text{mean}(|y|), \text{RMS}(y).$$

La pente lineaire est calculee par regression sur l'indice i :

$$a = \frac{\sum_i (i - \bar{i})(y_i - \bar{y})}{\sum_i (i - \bar{i})^2}.$$

Les differences donnent :

$$\sigma_{\Delta y} = \text{std}(y_{i+1} - y_i), \quad \max |\Delta y| = \max_i |y_{i+1} - y_i|.$$

Le taux de changement de signe est :

$$ZCR = \frac{1}{n-1} \sum_{i=1}^{n-1} \mathbf{1}(\text{sign}(y_{i+1}) \neq \text{sign}(y_i)).$$

L'autocorrelation au lag 1 est :

$$\rho_1 = \frac{\sum_i (y_i - \bar{y}_0)(y_{i+1} - \bar{y}_1)}{\sqrt{\sum_i (y_i - \bar{y}_0)^2} \sqrt{\sum_i (y_{i+1} - \bar{y}_1)^2}}.$$

19 Point important : qualite absolue versus qualite relative au cluster

Le calcul actuel produit :

$$S_{global} = \text{mean}(S_{signal}, S_{phys})$$

avant clustering. Ce score mesure une qualite absolue au sens des seuils de residus fixes.

Le clustering ne recalcule pas :

$$S_{global|cluster}$$

et ne produit pas de percentile ou de z-score robuste des residus dans le cluster.

Si l'objectif est de comparer une fenetre uniquement a des fenetres de dynamique similaire, une extension mathematiquement naturelle serait, pour chaque cluster C et chaque residu e :

$$z_{e,i}^{(C)} = \frac{e_i - \text{median}(e_j : j \in C)}{\max(1.4826 \text{ median}(|e_j - \text{median}_C(e)| : j \in C), \epsilon)}.$$

On pourrait ensuite definir un score relatif :

$$S_{cluster,i} = \frac{1}{1 + \text{mean}_{e \in \mathcal{E}} \max(z_{e,i}^{(C)}, 0)}$$

ou bien un percentile de qualite dans le cluster. Cette extension n'est pas presente dans le code actuel.